

Vinci Unitree — Day 3 Lecture Notes

B2 Industrial Fundamentals for a 3-Hour Instructor-Led Session

Author: Manus AI

Course: Vinci Unitree Train-the-Trainer

Session: Day 3 — B2 Industrial Fundamentals

Duration: 3 hours

Audience: Instructors and advanced students transitioning from Go2 fundamentals to Unitree B2 industrial operation

Brand note: Property of Vinci AI — Do Not Distribute. © 2026 Vinci AI. All rights reserved.

1. Session Purpose and Teaching Thesis

Day 3 is the transition point from **education-scale quadruped control** to **industrial-scale quadruped readiness**. The Unitree Go2 labs from Days 1–2 establish network setup, DDS communication, high-level motion APIs, ROS 2 vocabulary, inspection planning, and safety habits. Day 3 reuses those patterns, but the teaching emphasis changes because the B2 is heavier, stronger, higher-payload, and more operationally consequential. The B2 should therefore be taught not merely as a larger Go2, but as an industrial platform that requires stricter field discipline, clearer authority boundaries, and stronger pre-motion evidence.

Core teaching thesis: A B2 operator should never begin with motion. A competent B2 operator first proves network readiness, observes state, explains the control layer being used, identifies the safe stop path, and only then executes a supervised motion primitive.

The repository identifies Day 3 as **B2 Industrial Fundamentals**, with objectives covering B2 hardware, payloads, safety field protocols, navigation and control subsystem overview, and hands-on SDK scripting using the `unitree_go` IDL family. 1 The Day 3 lab sequence is intentionally conservative. Lab 0 is no-motion readiness, Lab 1 is no-motion state subscription, and Lab 2 introduces supervised sport RPC and stand behavior. 2 3 4

Teaching shift	Go2 teaching pattern	B2 Day 3 teaching pattern
Platform framing	Education and developer robot	Industrial quadruped for inspection, payload, endurance, and field deployment

First proof of readiness	Can the SDK talk to the robot?	Can the team prove interface, environment, DDS, state visibility, and safety perimeter before motion?
Motion philosophy	Small, bounded experimentation	Instructor-gated command sequence with explicit stop and recovery path
Student success evidence	Successful command execution	Safe diagnosis, correct interpretation of state, conservative operation, and clean stop discipline
Instructor role	API guide	Safety authority, run director, and escalation controller

By the end of the three-hour lecture, students should be able to explain why B2 training begins with observability, describe B2 hardware and sensor capabilities at a field-operator level, initialize the SDK communication environment, subscribe to B2 `SportModeState`, interpret the safety purpose of high-level `SportClient` commands, and distinguish high-level sport control from low-level motor command publishing.

2. Three-Hour Teaching Flow

The original course schedule labels Day 3 as a full-day B2 fundamentals scaffold. ⁶ This lecture note compresses the essential Day 3 learning outcomes into a three-hour session by combining concise lecture blocks, live code reading, no-motion validation, and a tightly supervised motion demonstration.

Time	Segment	Main instructor goal	Student output
00:00–00:15	Opening safety frame	Establish that B2 is an industrial robot, not simply a larger Go2	Students can state the stop authority and no-motion rule
00:15–00:40	B2 hardware and industrial context	Explain B2 size, payload, endurance, sensors, interfaces, and field implications	Students can identify why payload and speed change operating risk
00:40–01:05	B2 network and SDK readiness	Review interface naming, SDK environment, CycloneDDS	Students record the interface name and environment checks

		assumptions, and script locations	
01:05–01:30	Lab 1 state subscription	Run or dry-run <code>subscribe_sport_mode_state.py</code> and interpret state output	Students understand <code>rt/sportmodestate</code> as the pre-motion observability channel
01:30–01:40	Break and safety reset	Reset attention before any motion discussion	Students re-confirm perimeter and emergency actions
01:40–02:10	High-level SportClient control	Explain <code>Damp</code> , <code>BalanceStand</code> , <code>StopMove</code> , <code>StandUp</code> , <code>StandDown</code> , <code>RecoveryStand</code> , and limited movement commands	Students can explain which commands are allowed in Lab 2 and why
02:10–02:35	Supervised Lab 2 motion gate	Demonstrate <code>BalanceStand</code> and <code>StopMove</code> ; optionally discuss <code>b2_stand_example.py</code> without executing	Students can describe the difference between high-level and low-level control
02:35–02:50	Navigation, perception, and Day 4 bridge	Connect B2 sensors and camera scripts to inspection and analytics	Students can describe how B2 state and video become inspection data
02:50–03:00	Knowledge check and readiness sign-off	Confirm concepts, commands, safety gates, and next-day readiness	Students complete a verbal or written operator-readiness checklist

This flow deliberately puts **motion after state**. If a class is running on physical hardware, instructors should keep the first 90 minutes motion-free even if the robot is available. This creates the correct operational habit: communication, observability, and safety are not “setup chores” ; they are part of robot control.

3. B2 as an Industrial Platform

The Unitree B2 is positioned as an industrial quadruped for inspection, all-terrain operation, payload carrying, and long-duration field tasks. Unitree’s product page describes B2 as an industrial quadruped with a running speed greater than 6 m/s under special configurations and safety limits, joint torque around 360 N • m, standing load at least 120 kg, walking load greater than 40 kg, and unloaded endurance greater than five hours. ⁸ These values should not be presented to students as targets for classroom testing. They are specifications that explain why the training protocol must be conservative.

The B2 developer documentation lists a standing size of approximately 1098 mm × 450 mm × 645 mm, folded size of approximately 880 mm × 460 mm × 330 mm, approximate net weight of 60 kg including battery, 12 degrees of freedom, maximum speed greater than 6 m/s with safety limiting, IP67 protection, and operating temperature from -20°C to 55°C under good weather conditions. ⁷ The same documentation notes that B2 can be equipped with a 3D LiDAR, depth cameras, optical cameras, and multiple external interfaces. ⁷

B2 attribute	Instructor interpretation	Classroom implication
Approx. 60 kg robot mass	The platform has industrial inertia and can injure people or damage equipment	Treat motion as a controlled demonstration, not an exploratory exercise
Walking load >40 kg and standing load ≥120 kg	Payload capability is a defining industrial feature	Payload changes balance, clearance, current draw, and fall risk
Speed >6 m/s in special/safety-limited contexts	The robot has far more kinetic capability than required for training	Classroom motion should use only low-speed, instructor-approved primitives
4–6 h battery operating time	B2 is designed for extended deployment, not short toy demonstrations	Teach battery state and thermal checks as field-readiness evidence
IP67 protection and wide temperature range	The platform is intended for harsh environments	Classroom safety still matters; environmental rating is not permission for risky operation
LiDAR, depth, optical cameras	Perception supports inspection, mapping, and analytics	Day 3 should preview how state and video become Day 4 inspection data

A useful teaching move is to ask students to compare the same command across platforms. A `Move(0.5, 0, 0)` call on a small robot and a `Move(0.5, 0, 0)` call on a heavier industrial quadruped may look syntactically similar, but the field consequence is not the same. The

API similarity is helpful for learning, but it can be dangerous if it hides the physical difference.

4. B2 Safety Model for Training

The Day 3 labs begin with readiness and safety because the B2' s physical profile changes the risk model. Lab 0 explicitly requires confirmation of B2 network, SDK environment, CycloneDDS assumptions, and safety briefing before continuing. 2 This is the correct order: a student who cannot identify the network interface, terminal environment, stop command, and perimeter should not be allowed to run motion code.

Instructor rule: In B2 training, the first successful action is not walking. The first successful action is proving that the robot, operator, network, and safety perimeter are all in a known state.

The B2 developer documentation also warns that hot swapping aviation plug interfaces is strictly prohibited because it may cause equipment failure, and such failures are not covered under warranty. 7 This should be taught as a general field discipline principle: industrial robots combine computation, high-current power, communications, payloads, and mechanical motion, so cabling and interface handling are part of safety.

Safety gate	Required evidence	Instructor response if missing
Physical perimeter	Clear floor, no loose cables, no people in the motion zone, no fragile objects nearby	Do not run Lab 2 motion
Stop authority	One named instructor controls motion approval; one operator has keyboard focus; students know <code>StopMove</code> and emergency stop procedure	Pause and re-brief
Interface identity	Students can state the active network interface, such as <code>eth0</code> , <code>enp3s0</code> , or another verified wired adapter	Do not run scripts with guessed interfaces
Environment readiness	SDK environment is activated and expected variables are available	Fix environment before robot interaction
State observability	<code>SportModeState</code> prints for at least 30 seconds without DDS errors	Do not proceed to motion

Command scope	Only approved commands are allowed; initial commands are <code>BalanceStand</code> and <code>StopMove</code>	Stop session if students attempt unapproved motion
---------------	--	--

The instructor should distinguish **routine stop**, **motion stop**, and **emergency stop**. `StopMove` is a high-level motion stop that stops the current action and restores many internal motion parameters to defaults according to Unitree’s high-level motion documentation.¹¹ `Damp` enters a high-priority damping state in which motor joints stop moving and the robot enters a damping condition, and Unitree describes it as an emergency or unexpected-situation command in the high-level interface documentation.¹¹ Physical emergency procedures, remote control intervention, and site-specific emergency stop devices still supersede code-level convenience.

5. B2 System Architecture: From Wire to Motion

Day 3 should give students a simple but accurate mental model of the B2 control stack. Students do not need to become firmware developers, but they must know which layer they are touching when they run each script.

The Unitree SDK2 documentation describes the SDK as a package for Unitree’s newer robots that encapsulates interfaces for low-level motor control, high-level motion control, LiDAR point cloud data, audio and video transmission, SLAM, odometry, and other functions.⁹ The Unitree ROS2 repository explains that SDK2 uses a CycloneDDS-based communication mechanism supporting Go2, B2, and H1, and that because ROS 2 also uses DDS, ROS 2 messages can communicate with Unitree robots without wrapping the SDK interface.¹⁰

Layer	What students see	What it means in Day 3
Physical Ethernet	Cable, interface name, static network assumptions	The student must know which interface is connected to the robot
DDS communication	<code>ChannelFactoryInitialize(0, interface)</code>	The process joins the robot communication domain over the selected network interface
State topic	<code>rt/sportmodestate</code>	The robot publishes high-level motion state that should be observed before motion

High-level RPC	<code>SportClient()</code> calls such as <code>BalanceStand()</code> and <code>StopMove()</code>	The SDK sends semantic motion requests through a safety-managed high-level layer
Low-level command	<code>rt/lowcmd</code> and <code>rt/lowstate</code>	The script commands individual motor targets and requires stronger expertise
Perception/video	camera clients and RTSP streams	B2 can provide inspection data for later analytics and autonomy lessons

A central Day 3 concept is **control authority**. When students run `subscribe_sport_mode_state.py`, they are observing. When they run `b2_sport_client.py`, they are requesting high-level behaviors. When they run `b2_stand_example.py`, they are much closer to direct motor control, because the script publishes `LowCmd_` to `rt/lowcmd`, subscribes to `LowState_`, interpolates joint targets, and writes commands at a 0.002-second interval. These are not equivalent levels of risk.

6. Lab 0: B2 Readiness and Safety

Lab 0 is a no-motion lab. Its repository objectives are to confirm the B2 network, SDK environment, `CYCLONEDDS_HOME`, safety rules, and the B2 script location under `scripts/ives_sdk/B2/`. 2 The deliverable is simple: record the interface name and complete the safety briefing before Lab 1. 2

The instructor should resist the temptation to skip Lab 0. In professional robot deployment, readiness checks prevent false diagnoses. A failed motion script could be a network problem, an environment problem, a DDS problem, a robot state problem, a firmware mismatch, or a safety lock. Lab 0 narrows that uncertainty before the class interacts with the robot.

Lab 0 check	Example command or action	What success looks like
Repository location	<code>cd "\$(git rev-parse --show-toplevel)"</code>	The terminal is at the course repository root
SDK environment	<code>conda activate unitree_env</code>	The expected Python and SDK packages are available
Interface discovery	<code>ip link</code> or <code>ifconfig</code>	Students identify the wired robot interface

Script location	ls scripts/ives_sdk/B2/	Students see B2 scripts such as subscribe_sport_mode_state.py and b2_sport_client.py
Safety briefing	Instructor-led	Students can state perimeter, stop command, and command approval rule

The instructor can frame Lab 0 as a short operator certification. Students should be able to answer: Which interface are we using? Which environment is active? Which script observes state? Which script can move the robot? Who has stop authority? What is the first command we will use if motion does not look right?

7. Lab 1: Subscribe to B2 SportModeState

Lab 1 is the most important no-motion technical lab on Day 3. The repository instructs students to activate the SDK environment, move to the repository root, and run:

Bash

```
conda activate unitree_env
cd "$(git rev-parse --show-toplevel)"
python scripts/ives_sdk/B2/subscribe_sport_mode_state.py eth0
```

The success criterion is that state prints for at least 30 seconds without DDS errors. ³ Students should not treat this as a passive logging exercise. It is the first proof that the workstation, network interface, DDS communication, Python SDK imports, robot topic publication, and callback flow are working together.

The script imports ChannelFactoryInitialize and ChannelSubscriber, initializes the channel factory with the selected network interface, creates a subscriber on rt/sportmodestate, and receives SportModeState_. Unitree ROS2 documentation describes sport mode state as including position, velocity, IMU state, sport mode, gait type, body height, yaw speed, obstacle range, foot force, and foot positions/speeds in body frame. ¹⁰

State element	Why it matters	Instructor prompt
Position	Indicates estimated robot pose or motion-state position	“Does the value remain stable while the robot is stationary?”
Velocity	Reveals whether the robot believes it is moving	“Before motion, should velocity be near zero?”

IMU state	Indicates body orientation and inertial sensing	“What would a tilted or unstable robot look like in state?”
Mode	Indicates high-level motion mode such as idle, balance stand, locomotion, damping, recovery, or sit in Unitree’ s motion-state model	“Which mode would you expect before and after <code>BalanceStand</code> ?”
Gait type	Indicates gait category when locomotion is active	“Should gait change during a no-motion readiness check?”
Foot force	Helps diagnose contact and support	“What might asymmetric contact suggest?”
Foot position body	Shows leg geometry relative to body	“Why is this relevant before low-level control?”

Students should learn to interpret silence and errors as useful signals. If no state arrives, the class should not immediately rerun motion commands. They should verify the interface, cable, robot power, DDS environment, Python imports, and whether the robot publishes the expected topic. If state arrives but values are unreasonable, the correct response is to pause and inspect, not to proceed.

8. High-Level SportClient Commands

Lab 2 uses `b2_sport_client.py` , which exposes a terminal menu for high-level sport commands. The script constructs a `SportClient` , sets a 10-second timeout, initializes the client, and then maps menu choices to methods such as `Damp` , `BalanceStand` , `StopMove` , `StandUp` , `StandDown` , `RecoveryStand` , `Move` , `FreeWalk` , `ClassicWalk` , `MoveToPos` , and `TrajectoryFollow` . 5

The repository Lab 2 instruction is intentionally narrow: students should try **BalanceStand** and **StopMove** until the instructor approves more. 4 This is a strong instructional cue. The presence of a menu option does not imply permission to run it. The menu is a code artifact; the instructor gate is the operational authority.

Command	Teaching meaning	Recommended Day 3 status
<code>BalanceStand()</code>	Enter balanced standing behavior	Instructor-approved demonstration

<code>StopMove()</code>	Stop current motion and restore many motion parameters	Required stop command to know before motion
<code>Damp()</code>	Enter high-priority damping state	Discuss as emergency-related; use only by instructor policy
<code>StandUp()</code>	Locked tall standing posture	Discuss carefully; avoid prolonged locked posture
<code>StandDown()</code>	Locked low/lying posture	Instructor-only unless part of approved sequence
<code>RecoveryStand()</code>	Recover from fallen or lying state to standing	Instructor-only; important conceptually
<code>Move(vx, vy, vyaw)</code>	Body-frame velocity command	Not for first motion unless instructor explicitly approves
<code>ClassicWalk()</code> / <code>FreeWalk()</code>	Gait-related walking modes	Not for first three-hour Day 3 session unless site conditions are excellent
<code>MoveToPos()</code>	Relative/targeted position movement	Instructor-only; requires clear space and state confidence
<code>TrajectoryFollow()</code>	Follow future path points	Discuss but do not execute in basic Day 3; the script itself comments out the call

Unitree documentation describes `Move(vx, vy, vyaw)` as body-frame speed control and warns in the V2.0 motion documentation that the motion-control part does not filter `Move` commands and that the latest `Move` command is maintained for one second. The documentation recommends filtering before sending and sending `Move(0,0,0)` or `StopMove()` when stopping.¹² This should become a classroom rule: velocity commands are not “set and forget” ; they require stop discipline.

9. Lab 2: Supervised B2 Sport RPC and Stand

Lab 2 is the first motion lab. The repository labels it “Motion: yes — instructor gate” and lists `b2_sport_client.py` and `b2_stand_example.py` as the relevant scripts.⁴ The recommended run command for the sport menu is:

```
Bash
```

```
python scripts/ives_sdk/B2/b2_sport_client.py eth0
```

The class should not begin by exploring the menu. The instructor should define the exact sequence before the script starts. A safe introductory sequence is: confirm state subscription succeeded, clear the perimeter, run the script, list options if needed, issue `BalanceStand` , observe, issue `StopMove` , observe, then exit or return to a known posture under instructor direction.

Step	Instructor action	Student observation
1	Reconfirm no people or objects are inside the motion zone	Students visually inspect and verbally confirm
2	Confirm one operator has terminal focus	Students do not type commands independently
3	Start <code>b2_sport_client.py</code> with the verified interface	Students see the warning and menu prompt
4	Use <code>BalanceStand</code> only after approval	Students observe posture transition and state change
5	Use <code>StopMove</code>	Students observe stable stopping behavior
6	Discuss return code	Students learn that a return code is part of command evidence
7	Stop the demonstration	Students record commands executed and any anomalies

The optional `b2_stand_example.py` is not equivalent to the sport menu. It creates a low-command publisher on `rt/lowcmd` , a low-state subscriber on `rt/lowstate` , uses `MotionSwitcherClient` to release active modes, and writes motor commands every 0.002 seconds. It initializes 20 motor command slots but actively interpolates target positions for 12 leg motors, computes CRC, and publishes continuously. This is a powerful teaching example because it reveals the lower-level machinery under high-level behaviors, but it should be treated as instructor-only unless the class has already demonstrated strong low-level control readiness.

Classroom distinction: `SportClient` asks the robot to perform a behavior. `LowCmd` publishing tells individual motors what targets to pursue. These are different authority levels, different risk levels, and different debugging responsibilities.

10. Reading the B2 Scripts as Engineering Artifacts

A comprehensive Day 3 lecture should not merely run scripts. It should teach students how to read them. The B2 scripts encode several engineering patterns that students will reuse in later inspection and analytics tasks.

10.1 `subscribe_sport_mode_state.py`

This script is an example of **safe observability-first design**. It has no motion command path. Its main value is proving that the robot's state channel is reachable. The callback stores or prints the latest `SportModeState_`, and the main loop keeps the process alive until interrupted. Students should identify the three essential parts: initialize DDS, subscribe to the correct topic, and keep the program alive while callbacks arrive.

Python

```
ChannelFactoryInitialize(0, sys.argv[1])
subscriber = ChannelSubscriber("rt/sportmodestate", SportModeState_)
subscriber.Init(sport_state_callback, 10)
```

The code above is conceptually the B2 version of a medical monitor. It should be connected and understood before the intervention begins.

10.2 `b2_sport_client.py`

This script is an example of **high-level command routing**. It maps a user's menu choice to a method call. The key teaching point is that the code does not itself know the classroom safety plan. It exposes more options than students should use in the first session. Therefore, professional operation requires policy outside the code.

Python

```
sport_client = SportClient()
sport_client.SetTimeout(10.0)
sport_client.Init()
```

The timeout value matters because robot communication is not instantaneous or guaranteed. A failed or delayed RPC must be treated as a diagnostic event. Students should be taught to record the command, return code, robot state, and visible behavior.

10.3 `b2_stand_example.py`

This script is an example of **low-level periodic command streaming**. It subscribes to current low state, captures starting joint positions, interpolates toward target joint arrays, sets proportional and derivative gains, attaches CRC, and publishes low commands repeatedly.

Code feature	Engineering meaning	Why instructors should discuss it
<code>rt/lowcmd</code> publisher	Direct low-level command channel	Shows why low-level scripts require stronger authority
<code>rt/lowstate</code> subscriber	Feedback channel for motor state	Demonstrates closed-loop awareness, even in a simple example
<code>RecurrentThread(interval=0.002)</code>	500 Hz command-writing intent	Shows that low-level control is timing-sensitive
<code>Kp = 1000.0</code> , <code>Kd = 10.0</code>	Joint-control gains	Shows that stiffness and damping are explicit design choices
Target joint arrays	Desired leg configurations	Shows why joint numbering and limits matter
CRC computation	Message integrity check	Shows that robot command packets require validation

Students should come away understanding that low-level control is not “more advanced because it is cooler” ; it is more advanced because it removes abstraction and increases responsibility.

11. B2 Sensors, Video, and the Bridge to Inspection

Although Day 3 labs focus on readiness, state, and motion, the B2 platform should be framed as an inspection system. Unitree’ s B2 documentation states that the robot can include front and rear depth and optical cameras, plus a wide-angle omnidirectional LiDAR mounted at the head. 7 The product page also emphasizes applications such as power inspection, emergency rescue, and industrial inspection. 8

The repository’ s B2 script directory includes camera examples such as `camera_opencv-video.py` , `camera_opencv-videoEffect.py` , and `record_rtsp.py` . These scripts are not Day 3 lab deliverables, but they are valuable preview artifacts. They use front and back video clients, decode image data with OpenCV, support JPEG capture, and can record RTSP streams. One

variant adds simple image effects such as face detection, red color tracking, and sketch rendering. The instructor should present these scripts as a bridge to Day 4, where B2 inspection and analytics become more central.

Perception channel	Day 3 treatment	Day 4 bridge
<code>SportModeState</code>	Readiness and robot-state observability	Correlate robot state with inspection events
Optical cameras	Preview only	Image capture, video review, defect evidence, operator situational awareness
Depth cameras	Conceptual overview	Spatial context, obstacle awareness, inspection geometry
LiDAR	Conceptual overview	Mapping, localization, terrain and structure context
RTSP recording	Preview only	Evidence package for inspection report or analytics workflow

This is also the moment to teach that **inspection autonomy is data discipline**. The robot’s value is not only that it can move. Its value is that it can collect location-aware, time-aware, repeatable observations under field constraints.

12. Practical Troubleshooting Framework

Troubleshooting should be taught as a structured process rather than a sequence of guesses. The class should learn to move from physical layer to environment layer to communication layer to state layer to command layer.

Symptom	Likely layer	Diagnosis path	Safe response
Script cannot import SDK modules	Python environment	Confirm <code>conda activate unitree_env</code> ; verify installed SDK package	Do not connect motion commands until environment is correct

No state messages	Network or DDS	Check cable, interface name, robot power, DDS configuration, and topic name	Stay in Lab 1; do not proceed to Lab 2
DDS errors appear	DDS configuration or interface conflict	Verify only intended network interface is selected; check CycloneDDS assumptions	Pause and fix configuration
State prints but values are unexpected	Robot state or interpretation	Compare stationary robot velocity, mode, posture, and visible state	Ask instructor before motion
Sport command returns nonzero	RPC or robot mode	Record command, return code, current state, and visible behavior	Stop; do not chain commands
Robot begins unexpected motion	Command or mode issue	Use approved stop path; instructor takes control	Clear perimeter and investigate after stop
Low-level script behaves unexpectedly	Timing, gains, state, mode, or command stream	Do not debug live near people	Instructor-only recovery and code review

A useful instructor phrase is: **“No silent failures.”** Every failure should produce a short incident note: time, command, interface, visible robot state, terminal output, action taken, and whether the robot returned to a known safe state.

13. Instructor Script for the 3-Hour Lecture

The following narrative can be used directly during instruction.

Opening narrative

“Today we move from Go2-scale training to B2 industrial fundamentals. The code patterns will look familiar: we still use a network interface, DDS, Unitree SDK calls, and state subscriptions. However, the physical consequences are different. The B2 is an industrial quadruped with significant mass, payload capacity, endurance, and sensing capability. Therefore, our first objective is not to make it walk. Our first objective is to prove that we can observe it, reason about it, and stop it.”

Readiness narrative

“Before any motion command, we must identify the interface, activate the SDK environment, confirm the script location, and run a no-motion state subscriber. If state does not print cleanly for 30 seconds, we do not proceed. A robot that cannot be observed should not be commanded.”

State narrative

“ `SportModeState` is our first diagnostic window. It tells us the robot’s high-level motion state, velocity, body height, gait state, and other motion-related information. We will not memorize every field today. Instead, we will build the habit of checking whether the robot’s reported state matches the visible robot.”

Motion narrative

“The sport menu exposes many commands. That does not mean the class is authorized to run all of them. Today our first motion commands are `BalanceStand` and `StopMove`, and only after the instructor gates the demonstration. We will observe the return code and visible behavior. If anything looks wrong, we stop and diagnose.”

Low-level narrative

“The stand example is different from the sport client. It publishes low-level commands to motor targets at a fast interval. This is valuable for understanding the robot, but it is not a beginner exploration tool. The difference between high-level behavior request and low-level motor command is one of the most important distinctions in robot safety.”

14. Student Knowledge Checks

Students should answer these questions at the end of the session. The instructor may use them verbally, as a written handout, or as a lab sign-off checklist.

Check	Expected answer quality
Why does Day 3 begin with a no-motion readiness lab?	Student explains that B2 motion must be preceded by network, environment, DDS, observability, and safety checks.
What is the purpose of <code>rt/sportmodestate</code> ?	Student explains that it provides high-level robot motion state used to verify readiness and interpret robot behavior.

Why is <code>BalanceStand</code> safer as a first demonstration than <code>Move</code> ?	Student explains that it demonstrates controlled posture without intentionally commanding translation.
What should happen if <code>subscribe_sport_mode_state.py</code> does not print state cleanly?	Student says to stop at diagnosis and not proceed to motion.
What is the difference between <code>SportClient</code> and publishing to <code>rt/lowcmd</code> ?	Student identifies high-level behavior requests versus low-level motor command streaming.
Why is <code>Move(0,0,0)</code> or <code>StopMove()</code> part of command hygiene?	Student explains that velocity commands require explicit stopping and Unitree documentation warns about command filtering and hold behavior.
What are the main B2 sensing channels relevant to future inspection work?	Student names LiDAR, depth cameras, optical cameras, video/RTSP, and robot state.
What is the instructor gate in Lab 2?	Student states that only approved commands may run, initially <code>BalanceStand</code> and <code>StopMove</code> .

15. Day 3 Takeaways

Day 3 should leave students with a disciplined mental model of industrial quadruped operation. B2 is powerful because it combines mobility, payload, endurance, sensing, and SDK access. Those same characteristics require stronger safety habits than a small classroom robot. The central lesson is not that B2 can move; the central lesson is that a trained operator can decide when it should move, verify that it is ready, observe its state, issue a bounded command, and return it to a safe condition.

The instructor should emphasize four durable habits. First, **observe before acting** by using `SportModeState` . Second, **gate motion** through a named instructor and approved command sequence. Third, **know the control layer** before running code, especially the difference between high-level `SportClient` calls and low-level `LowCmd` streaming. Fourth, **treat inspection as data discipline**, where state, video, timing, and operator notes become the foundation for Day 4 analytics.

Final concept	One-sentence student memory
Readiness	"If I cannot prove the interface and state channel, I do not command motion."

Safety	“The robot moves only inside an instructor-approved command envelope.”
Observability	“ <code>SportModeState</code> is my first diagnostic window into B2 behavior.”
Control layers	“High-level sport commands request behavior; low-level commands stream motor targets.”
Inspection bridge	“B2 becomes valuable when motion, state, video, and field evidence are connected.”

16. References

- [1] Vinci-AI-Analytics — Day 3: B2 Industrial Fundamentals
- [2] Vinci-AI-Analytics — Lab 0: B2 Readiness & Safety
- [3] Vinci-AI-Analytics — Lab 1: Subscribe to B2 SportModeState
- [4] Vinci-AI-Analytics — Lab 2: B2 Sport RPC & Stand
- [5] Vinci-AI-Analytics — B2 Sport Client Script
- [6] Vinci-AI-Analytics — TtT Schedule Map
- [7] Unitree Robotics — B2 SDK Development Guide: About B2
- [8] Unitree Robotics — Unitree B2 Product Page
- [9] Unitree Robotics — Obtain SDK
- [10] Unitree Robotics — unitree_ros2
- [11] Unitree Robotics — Sports Services Interface
- [12] Unitree Robotics — Motion Services Interface V2.0